

SKILL 2

```
1  -- 0. CLEANUP (Ensures a fresh, clean execution)
2  • DROP DATABASE IF EXISTS gayathri_2520080010;
3
4  -- 1. CREATE DATABASE AND TABLE
5  • CREATE DATABASE gayathri_2520080010;
6  • USE gayathri_2520080010;
7
8  • CREATE TABLE orders (
9      ord_no INT PRIMARY KEY,
10     purch_amt DECIMAL(10,2),
11     ord_date DATE,
12     customer_id INT,
13     salesman_id INT
14 );
15
16 -- 2. INSERT DATA
17 • INSERT INTO orders (ord_no, purch_amt, ord_date, customer_id, salesman_id) VALUES
18 (70001, 150.50, '2012-10-05', 3005, 5002),
19 (70009, 270.65, '2012-09-10', 3001, 5005),
20 (70002, 65.26, '2012-10-05', 3002, 5001),
21 (70004, 110.50, '2012-08-17', 3009, 5003),
22 (70007, 948.50, '2012-09-10', 3005, 5002),
23 (70005, 2400.60, '2012-07-27', 3007, 5001),
24 (70008, 5760.00, '2012-09-10', 3002, 5001),
25 (70010, 1983.43, '2012-10-10', 3004, 5006),
26 (70003, 2480.40, '2012-10-10', 3009, 5003),
27 (70012, 250.45, '2012-06-27', 3008, 5002),
28 (70011, 75.29, '2012-08-17', 3003, 5007),
29 (70013, 3045.60, '2012-04-25', 3002, 5001);
30
31 -- 3(1). WHERE CLAUSE - Orders over 2000
32 • SELECT *
33 FROM orders
34 WHERE purch_amt > 2000;
35
36 -- 3(2). WHERE CLAUSE - Specific date
37 • SELECT *
38 FROM orders
39 WHERE ord_date = '2012-09-10';
40
41 -- 3(3). WHERE CLAUSE - Specific salesman
42 • SELECT *
43 FROM orders
44 WHERE salesman_id = 5001;
45
46 -- 4. ORDER BY - Purchase amount descending
47 • SELECT *
48 FROM orders
49 ORDER BY purch_amt DESC;
```

```
46 -- 4. ORDER BY - Purchase amount descending
47 • SELECT *
48 FROM orders
49 ORDER BY purch_amt DESC;
50
51 -- 5. ORDER BY - Order date ascending
52 • SELECT *
53 FROM orders
54 ORDER BY ord_date;
55
56 -- 6. SUM()
57 • SELECT SUM(purch_amt) AS total_revenue
58 FROM orders;
59
60 -- 7. AVG()
61 • SELECT AVG(purch_amt) AS average_order
62 FROM orders;
```

```
64 -- 8. MAX()
65 • SELECT MAX(purch_amt) AS highest_order
66 FROM orders;
67
68 -- 9. MIN()
69 • SELECT MIN(purch_amt) AS lowest_order
70 FROM orders;
71
72 -- 10. COUNT()
73 • SELECT COUNT(*) AS total_orders
74 FROM orders;
75
76 -- 11. GROUP BY - Total sales by salesman
77 • SELECT salesman_id,
78        SUM(purch_amt) AS total_sales
79 FROM orders
80 GROUP BY salesman_id;
```

```
76 -- 11. GROUP BY - Total sales by salesman
77 • SELECT salesman_id,
78        SUM(purch_amt) AS total_sales
79 FROM orders
80 GROUP BY salesman_id;
81
82 -- 12. GROUP BY - Total purchase by customer
83 • SELECT customer_id,
84        SUM(purch_amt) AS total_purchase
85 FROM orders
86 GROUP BY customer_id;
87
88 -- 13. GROUP BY + MAX()
89 • SELECT customer_id,
90        MAX(purch_amt) AS highest_purchase
91 FROM orders
92 GROUP BY customer_id;
```

```

94 -- 14. GROUP BY + HAVING - Salesman total > 3000
95 • SELECT salesman_id,
96       SUM(purch_amt) AS total_sales
97 FROM orders
98 GROUP BY salesman_id
99 HAVING SUM(purch_amt) > 3000;
100
101 -- 15. GROUP BY + HAVING - Customer total > 2500
102 • SELECT customer_id,
103       SUM(purch_amt) AS total_purchase
104 FROM orders
105 GROUP BY customer_id
106 HAVING SUM(purch_amt) > 2500;
107
108 -- 16. GROUP BY + HAVING - Customer order count > 1
109 • SELECT customer_id,
110       COUNT(*) AS total_orders
111 FROM orders

```

```

109 • SELECT customer_id,
110       COUNT(*) AS total_orders
111 FROM orders
112 GROUP BY customer_id
113 HAVING COUNT(*) > 1;
114
115 -- 17. GROUP BY + HAVING + ORDER BY
116 • SELECT customer_id,
117       SUM(purch_amt) AS total_purchase
118 FROM orders
119 GROUP BY customer_id
120 HAVING SUM(purch_amt) > 1000
121 ORDER BY total_purchase DESC;
122
123 -- 18. BETWEEN + HAVING
124 • SELECT customer_id,
125       MAX(purch_amt) AS max_purchase
126 FROM orders

```

```

126 FROM orders
127 GROUP BY customer_id
128 HAVING MAX(purch_amt) BETWEEN 2000 AND 6000;
129
130 -- 19. COUNT + HAVING - Salesman with more than 2 orders
131 • SELECT salesman_id,
132       COUNT(*) AS total_orders
133 FROM orders
134 GROUP BY salesman_id
135 HAVING COUNT(*) > 2;
136
137 -- 20. MAX + GROUP BY + HAVING - Highest purchase per date > 2000
138 • SELECT ord_date,
139       MAX(purch_amt) AS highest_purchase
140 FROM orders
141 GROUP BY ord_date
142 HAVING MAX(purch_amt) > 2000;

```

Result Grid		Filter Rows:	Exports:	Wrap Cell Content:
customer_id	highest_purchase			
3005	948.50			
3002	5760.00			
3009	2480.40			
3007	2400.60			
3001	270.65			
3004	1983.43			
3003	75.29			
3008	250.45			

Result 7	Result 8	Result 9	Result 10	Result 11	Result 12	Result 13	Result 14	Result 15	Result 16	Result 17	Result 18	Result 19	Result 20	Read Only	Context Help	Snippets
----------	----------	----------	-----------	-----------	-----------	-----------	-----------	-----------	-----------	-----------	-----------	-----------	-----------	-----------	--------------	----------

Output	
Action Output	
#	Time
Action	Message
Duration / Fetch	
61	10:57:03
SELECT customer_id,	SUM(purch_amt) AS total_purchase FROM orders GROUP BY customer_id HAVIN...
62	10:57:03
SELECT customer_id,	COUNT(*) AS total_orders FROM orders GROUP BY customer_id HAVING COUN...
63	10:57:03
SELECT customer_id,	SUM(purch_amt) AS total_purchase FROM orders GROUP BY customer_id HAVIN...
64	10:57:03
SELECT customer_id,	MAX(purch_amt) AS max_purchase FROM orders GROUP BY customer_id HAVIN...
65	10:57:03
SELECT salesman_id,	COUNT(*) AS total_orders FROM orders GROUP BY salesman_id HAVING COUN...
66	10:57:03
SELECT ord_date,	MAX(purch_amt) AS highest_purchase FROM orders GROUP BY ord_date HAVING ...